

JP ADMIN

Self-Hosted Installation Guide

Discord + GitHub + Render + Google Sheets + Apps Script

A calm, step-by-step guide for mentors who do not write code

Version 2.0 | September 2026

THE PROMISE

Follow the numbered steps in order. Every checkpoint tells you what success looks like. Setup reuses existing Discord channels and preserves messages, Sheet tabs, student rows, and tracking history.

Estimated first setup time: 30-45 minutes. Keep this PDF open in one tab while you work in the other services.

Your setup route

You will create five connected pieces. Each belongs to you and is independent from every other mentor.

Order	Piece	Purpose
1	Private GitHub copy	Your bot source; Render watches its main branch
2	Discord application	Your bot identity, token, intents, and server installation
3	Render service	Runs the bot from your private repository
4	Google Sheet + Apps Script	Stores cohort data and performs backend work
5	Private #bot-admin setup	Matches channels, syncs students, and verifies health

STOP AND CHECK SECRETS

A Discord Application ID is public. A Discord bot token, COHORT_API_KEY, and Apps Script SECRET_KEY are passwords. Never put passwords in GitHub, Discord chat, screenshots, issues, or this PDF.

Before you begin

- [] You own or can administer the Discord server.
- [] You have a Google account that will own the cohort Sheet.
- [] You have GitHub and Render accounts.
- [] You have the JP ADMIN v2.0 public repository or release ZIP.
- [] You can keep this guide open until the final green checks pass.

NO-DATA-LOSS RULE

Do not delete existing channels, roles, Sheet tabs, rows, or messages when something fails. Use the Retry button or the recovery table near the end of this guide.

STEP 1

Create your private GitHub copy

DONE when `render.yaml` is visible at the repository top level

Recommended method: Use this template

1. Open github.com/sasohan0/jp-admin-self-hosted.
2. Choose **Use this template**, then **Create a new repository**.
3. Choose your own GitHub account as Owner.
4. Name it something clear, such as `jp-admin-my-cohort`.
5. Choose **Private**. Do not add another README, `.gitignore`, or license.
6. Create the repository and confirm `render.yaml`, `package.json`, and `index.js` are visible at the top level.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner * Repository name *

sasohan0 /

Great repository names are short and memorable. How about [fuzzy-happiness?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository

Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Example GitHub new-repository screen. Use your own account, cohort name, and Private visibility.

If you downloaded the ZIP instead

IMPORTANT

Extract the ZIP first. Upload the extracted files, not the ZIP file itself. Avoid a nested folder: `render.yaml` must be at the repository top level.

Easy upload with GitHub Desktop

1. Create an empty **Private** GitHub repository.
2. Install and sign in to GitHub Desktop.
3. Choose **File - Clone repository** and clone the empty repository.
4. Copy every extracted package file into the cloned folder.
5. In GitHub Desktop enter summary `Initial JP ADMIN setup`, choose **Commit to main**, then **Push origin**.

Command-line alternative

```
git init
git add .
git commit -m "Initial JP ADMIN setup"
git branch -M main
git remote add origin https://github.com/YOUR-NAME/YOUR-REPOSITORY.git
git push -u origin main
```

CHECKPOINT 1

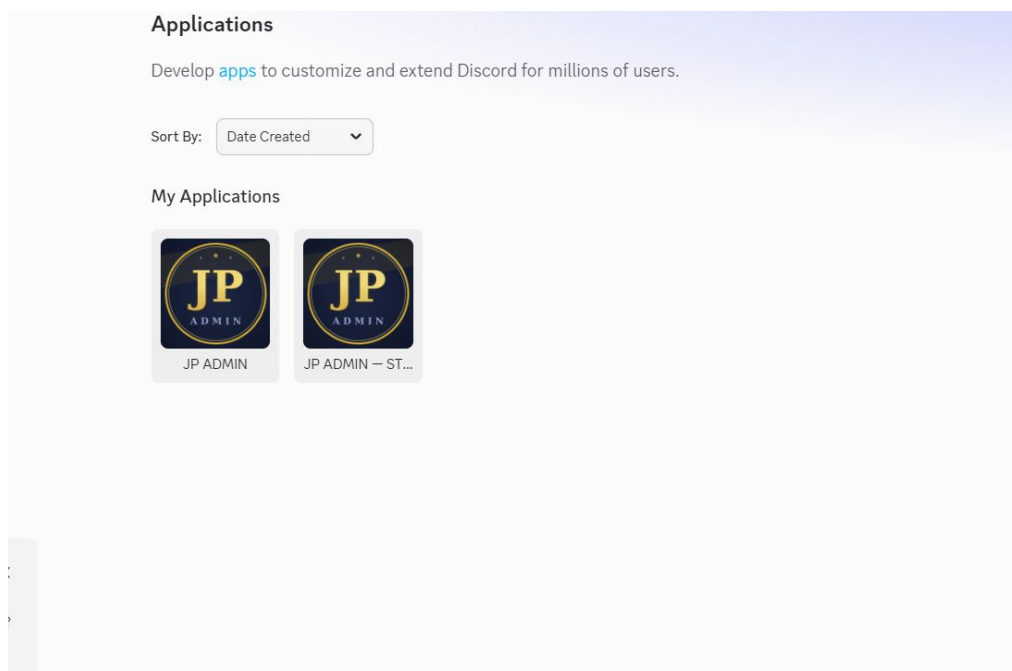
Open the private repository in your browser. If `render.yaml` is not visible immediately, fix the folder structure before continuing.

STEP 2

Create the Discord application

DONE when you have a private token and both privileged intents are ON

1. Open the [Discord Developer Portal](#).
2. Choose **New Application** and name it JP ADMIN - Your Cohort.
3. Open **Bot**. Choose **Reset Token**, then copy the token into a temporary private note.
4. Under **Privileged Gateway Intents**, enable **Server Members Intent**.
5. Enable **Message Content Intent** and save changes.



Discord Developer Portal. Create and open your own application; never share its bot token.

IF THE TOKEN LEAKS

Reset it immediately in Discord and later replace only DISCORD_TOKEN in Render. A token is not the Application ID.

STEP 3

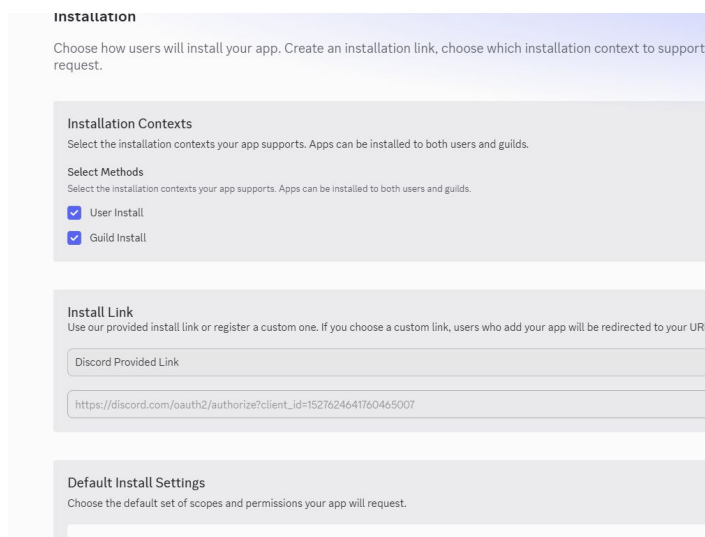
Install the bot in your Discord server

DONE when the bot appears as a server member and /setup is available

DO NOT MISS THESE TWO SCOPES

Select both bot and applications.commands. Selecting only bot can prevent /setup from appearing.

1. In the Developer Portal open **Installation** and enable **Guild Install**.
2. Under Guild Install scopes select bot.
3. Also select applications.commands.
4. Under bot permissions select **Administrator**.
5. Open the generated install link, choose the correct server, and approve.
6. In Discord open **Server Settings - Roles**. Move the JP ADMIN bot role above all student status, identity, readiness, inactive, and hired roles it will manage.



Discord Installation page. Scroll to Default Install Settings and confirm both scopes plus Administrator before using the link.

Exact fallback invite URL

```
https://discord.com/oauth2/authorize?
client_id=YOUR_APPLICATION_ID&permissions=8&
scope=bot%20applications.commands
```

Replace only YOUR_APPLICATION_ID with the public Application ID from General Information. Never put the bot token in a URL.

STEP 4

Deploy your private copy on Render

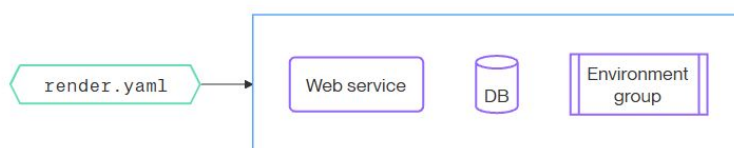
DONE when Render says Live and /health returns HTTP 200

1. Open [Render Blueprints](#).
2. Choose **New Blueprint Instance** and connect the private GitHub repository from Step 1.
3. Confirm Render detected `render.yaml`.
4. When Render asks for `DISCORD_TOKEN`, paste the private Discord token into Render's secret field.
5. Create the Blueprint and wait for the web service to show **Live**.
6. Open `https://YOUR-SERVICE.onrender.com/health`. Expect HTTP 200. During first setup, installer/waiting status is normal.

Render Blueprints (IaC)

Manage your Render infrastructure with a single YAML file.

Blueprints are Render's infrastructure-as-code (IaC) model for defining, deploying, and managing multiple resources with a single YAML file:



[+ Show example Blueprint](#)

A Blueprint acts as the single source of truth for configuring an interconnected set of services, databases, and [environment groups](#). Whenever you update a Blueprint, Render automatically redeploys any affected services to apply the new configuration (you can [disable this](#)).

As your infrastructure grows over time, Blueprints become more and more helpful for managing changes and additions to it.

Render Blueprint screen. Connect your private repository and supply the token only in Render.

What the Blueprint sets automatically

Setting	What happens	Do you type it now?
DISCORD_TOKEN	Secret used to sign in to your Discord bot	Yes - Render asks once
JP_INSTALLER_MODE	Starts safe first-run setup instead of a legacy cohort	No - automatic
COHORT_API_KEY	Creates the private backend key	No - generated
COHORT_API_URL	Starts with a temporary placeholder	Replace after Apps Script deploy
BOT_ACTIVE_WINDOW	Keeps the configured bot window continuous	No - automatic

Deploy for Free

Preview the Render platform with free web services and datastores.

You can deploy certain Render service types **free of charge**:

- Web services (web apps in Node.js, Python, Rails, etc.)
- Render Postgres databases
- Render Key Value instances

Free instances have important limitations, described below. Do not use them for production applications. However, they're perfect for testing out a new technology, working on a hobby project, or previewing Render's developer experience.

You can also deploy static sites on Render for free.

i Web services and static sites count against your monthly included amounts of outbound bandwidth and pipeline minutes. View your workspace's monthly included usage in the Render Dashboard.

[hangelog](#) [Ask AI](#)

Render free-service information. Free services can sleep and take time to wake.

FREE PLAN EXPECTATION

The Blueprint can use a free web service. Free instances may sleep, restart, or wake slowly. Google Sheets holds durable cohort data; Render's local disk is not the system of record. For continuous availability, an always-on paid instance is safer.

STEP 5

Open the private Discord setup assistant

DONE when the four-button panel appears in private #bot-admin

1. Wait until Render is Live, then return to the Discord server.
2. Type `/setup` and choose the JP ADMIN command. This is the recommended first command.
3. JP ADMIN reuses an existing #bot-admin or creates it only if missing.
4. The bot repairs privacy so ordinary members cannot view the channel.
5. Continue only in private #bot-admin.

OWNER RECOVERY IS AUTOMATIC

The Discord server owner is always saved as the permanent recovery supervisor. If another administrator starts setup, that administrator is saved too. The owner cannot be removed, preventing a bot-admin blackout.

If `/setup` does not appear

- 1 Reinstall the bot using both OAuth scopes: bot and applications.commands.
- 2 Wait up to one minute, reopen Discord, and type `/` again.
- 3 If `!setup` alone is silent, enable Message Content Intent and restart Render.
- 4 If the bot is offline, check Render Live status, `/health`, and Render logs.

SAFE RETRY

Running `/setup` again refreshes the panel. It does not delete channels, messages, Sheet data, or students.

STEP 6

Create and authorize the Google backend

DONE when authorizeAllRequiredServices and setup both finish

1. Create or open the Google Sheet for this cohort. Existing student rows may stay.
2. Choose **Extensions - Apps Script**.
3. Delete only the empty starter function. Paste the complete Code - v19 - FINAL . gs file from the package.
4. Near the top edit only COHORT, TZ, and SECRET_KEY.
5. For SECRET_KEY, copy the generated COHORT_API_KEY from Render - Environment. Paste it only in your private Apps Script project.
6. Save. Select authorizeAllRequiredServices and press **Run**.
7. Choose Review permissions, select the Sheet owner's Google account, use Advanced if shown, and allow the requested access.
8. Select setup and run it once.



Apps Script function selector. Run authorizeAllRequiredServices before deploying the Web App.

WHAT GOOGLE AUTHORIZATION DOES

It checks Spreadsheet, Forms, triggers, mail quota/Gmail access, and outbound requests. The helper sends no student email and deletes no data.

STEP 7

Deploy Apps Script and connect Render

DONE when the setup button reports the backend is reachable

Deploy the Web App

1. In Apps Script choose **Deploy - New deployment**.
2. Select type **Web app**.
3. Set **Execute as: Me**.
4. Set **Who has access: Anyone**.
5. Deploy and copy the final URL ending in `/exec`.
6. In Render open your service - **Environment**.
7. Replace only `COHORT_API_URL` with the `/exec` URL. Save changes.
8. Wait for Render to redeploy and show Live again.

URL RULE

For future Apps Script code updates, edit the existing Web App deployment to a new version. Keep the same `/exec` URL. Do not create a second public URL just because a setup step is retried.

Return to Discord

Run `/setup` again in private `#bot-admin`. Press **1 - Google permissions**, then **Done - test connection**. A successful response should say Apps Script responded safely.

IF THE TEST FAILS

Check three items only: the URL ends in `/exec`, Web App access is Anyone, and Apps Script `SECRET_KEY` exactly matches Render `COHORT_API_KEY`. Fix the named item and press Retry.

STEP 8**Match, sync, and verify**

DONE when private diagnostics pass without student pings

Button	What it does	Safe result
1 - Google permissions	Shows authorization instructions and tests the backend	Read-only connection test
2 - Match channels	Uses configured IDs or recognized names, then creates only missing channels	Existing channels/messages stay
3 - Sync students	Captures every eligible current Discord member, even without intake	No existing student record is deleted
4 - Verify	Checks backend health and protected-channel permissions	Private actionable report

Run the final private commands

```
!checkperms
!doctor
!syncmembers
```

DO NOT PING STUDENTS

These diagnostics suppress mentions. They belong in private #bot-admin. Required backend, permission, roster, and schedule checks should pass; an intentionally disabled optional AI/Groq feature can remain disabled.

Success looks like

- [] Render /health returns HTTP 200.
- [] Discord shows the bot online.
- [] /setup opens one panel in #bot-admin.
- [] !checkperms reports protected channels checked.
- [] !doctor reports the backend, roster, permissions, and schedules healthy.
- [] !syncmembers reports existing Discord students, including those without intake.

How existing servers and students are protected

Existing item	JP ADMIN behavior
Discord channels	Reuses valid configured IDs first, then recognized channel names; creates only missing channels
Messages and pins	Preserves history; setup adds its own private signed state message
Roles	Repairs bot and supervisor access; never assumes Administrator bypasses role hierarchy
Current students without intake	Creates stable provisional tracking identities during Sync students
Existing Sheet tabs/rows	Repairs required structure and archives ambiguous mappings instead of deleting history
Multiple cohorts	A mentor-owned self-hosted copy is paired to one server and its own backend

WHEN NAMES LOOK DUPLICATED

Do not delete channels or Sheet rows. Run `!ensurechannels` or `!checkperms`, inspect the result, and configure the intended channel only after identifying it.

Add other supervisors after backend connection

```
!supervisor list
!supervisor add @mentor
!supervisor remove @mentor
```

Run these commands only in private `#bot-admin`. Self-hosted changes are saved immediately in the signed setup capsule and do not need a Render restart. The server owner remains the recovery supervisor.

First-day operating checklist

Installation is complete before any student-facing reports are posted. Use this page as the first mentor handoff.

Purpose	Start with	Important effect
Health	!doctor	Private; no student ping
Permissions	!checkperms	Private; no student ping
Students	!syncmembers	Refreshes durable roster/tracking
Attendance	!formstatus, then !openform / !closeform	Open/close posts are student-facing
Job tracker audit	!checkjobsheets YYYY-MM-DD	Private read-only; no score write
Combined audit	!checkpipelines YYYY-MM-DD	Private diagnostic
Leave review	!openleaves	Paged private approval manager
Weekly report	!weeklyreport	Posts the performance report
Settings	!control	Private overview
All commands	!help	Private categorized command center

READ BEFORE DAILY USE

Open MENTOR_OPERATIONS_GUIDE.md for attendance, job tracking, weekly reports, leave decisions, backfills, and safe recovery. MENTOR_COMMAND_REFERENCE.md contains the complete categorized command list.

Fast troubleshooting without frustration

Find the symptom, perform only the listed recovery, and retry the same checkpoint. Do not delete data.

Symptom	Safe recovery
Bot not in server	Repeat OAuth install with bot + applications.commands and Administrator
Bot offline	Check Render Live, /health, then Render logs
/setup missing	Reinstall both OAuth scopes; wait one minute; reopen Discord
!setup silent	Use /setup; enable Message Content Intent; restart Render
Not a supervisor	Server owner runs /setup, then !supervisor add @mentor
Discord login error	Reset token; replace only DISCORD_TOKEN in Render
Backend test fails	Verify /exec, Anyone access, and matching private key
Students missing	Enable Server Members Intent; run !syncmembers privately
Permission issue	Move bot role higher; run !checkperms then !repairpermissions
Render free service asleep	Open /health and wait for wake; inspect logs; consider always-on plan
Duplicate-looking output	Pause relevant automation in !control; inspect schedule before rerun

WHEN ASKING FOR HELP

Share only the exact error text after removing student information. Never share the Discord token, COHORT_API_KEY, SECRET_KEY, private Sheet contents, phone numbers, resumes, or private screenshots.

Safe updates and service recovery

Updating your private bot

1. Download the new public release or compare the public repository.
2. Copy changed source files into your private GitHub repository.
3. Do not replace Render environment values or Apps Script CONFIG secrets.
4. Commit to main and push. Render redeploys the connected branch automatically.
5. If Code-v19-FINAL.gs changed, update the existing Apps Script Web App deployment to a new version while keeping its /exec URL.
6. After deployment run /setup, !checkperms, !doctor, and the relevant private audit.

Temporary local failover

For a regional Render outage, a trusted local computer can use the guarded failover included in the package. Run `npm ci`, copy `.env.failover.example` to the gitignored `.env.failover`, fill it only with this deployment's values, then run `npm run start:failover`.

FAILOVER SAFETY

The guard refuses the wrong cohort or bot, refuses to start while Render is healthy, and stops the local process after Render recovers. Never run the same Discord token in two healthy bot processes.

Final completion card

Do not call installation finished until every required box below is checked.

- [] Private GitHub repository contains render.yaml at the top level.
- [] Discord has Server Members Intent and Message Content Intent enabled.
- [] OAuth installation used bot + applications.commands + Administrator.
- [] Bot role is above every role it must manage.
- [] Render is Live and /health returns HTTP 200.
- [] /setup created or reused a private #bot-admin.
- [] Server owner is listed by !supervisor list.
- [] authorizeAllRequiredServices and Apps Script setup completed.
- [] Apps Script Web App executes as Me, allows Anyone, and URL ends in /exec.
- [] Render COHORT_API_URL contains that /exec URL.
- [] All four setup buttons completed successfully.
- [] !checkperms, !doctor, and !syncmembers passed privately.
- [] Existing channels, messages, Sheet data, and students were preserved.

YOU ARE READY

Keep this PDF, MENTOR_OPERATIONS_GUIDE.md, and the bot's private !help command available to every supervisor. Add supervisors with !supervisor add @mentor only after the backend connection passes.

Public source and updates

<https://github.com/sasohan0/jp-admin-self-hosted>

This guide contains no token or backend secret. Each mentor must create and protect their own credentials.